



Object Oriented Programming

Pass Task 4.2: Case Study — Iteration 2: Players, Items, and Inventory

Overview

Object-oriented programming makes best sense with larger programs. The case study will be your opportunity to create a larger program and better understand how the object-oriented approach can make it easier to create complex software solutions.

- Purpose:** Demonstrate the use of inheritance and polymorphism in the case study.
- Task:** Understand the case study program and implement iteration 2.
- The task contains personalized requirements**
- Deadline:** Due by the start of week five, **Fri, 30 August 2024, 23:59:00** (Firmed).

Submission Details

All students have access to the Adobe Acrobat tools. Please print your solution to PDF and combine it with the screenshots taken for this task.

- Program source code
- Test source code
- Screenshot of unit tests passing

Instructions

1. Review the *Case Study Requirements* document. It outlines what you need to create.
2. For this week aim to complete Iteration 2.

Note: At this point there will not be a "program" as such, just a set of unit tests that help demonstrate that your solution is moving towards completion.

Once your tests are working correctly get a screenshot of the tests passing and submit them along with the code.

Assessment Criteria

Make sure that your task has the following in your submission:

- The program is implemented correctly based on the case study description.
- The “Universal Task Requirements” (see Canvas) have been met.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class IdentifiableObject
    {
        private List<string> _Identifiers;

        public IdentifiableObject(string[] idents)
        {
            _Identifiers = new List<string>(idents);
        }
        public bool AreYou(string id)
        {
            return _Identifiers.Contains(id);
        }
        public string FirstId
        {
            get
            {
                return _Identifiers[0];
            }
        }
        public string AddIdentifier(string _Identifiers)
        {
            _Identifiers.ToLower();
            return _Identifiers;
        }
    }
}
```

```

    }
    public void PrivilegeEscalation(string pin)
    {
        string studentid = "103995219";
        if (studentid.Substring(studentid.Length - 4) == pin)
        {

            _Identifiers[0] = "8";
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Swin_Adventure2
{
    public class GameObject : IdentifiableObject
    {
        private string _name;
        private string _description;

        public GameObject(string[] ids, string name, string desc) : base(ids)
        {
            _name = name;
            _description = desc;
        }
        public string Name
        {
            get
            {
                return _name;
            }
        }
        public string ShortDescription
        {
            get
            {
                return $"{_name} ({FirstId})";
            }
        }
        public virtual string FullDescription
        {
            get
            {
                return _description;
            }
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Swin_Adventure2
{
    public class Item : GameObject
    {
        public Item(string[] idents, string name, string desc) : base(idents, name, desc)
        {
        }
    }
}

```

```

}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class Inventory
    {
        private List<Item> _items;

        public Inventory()
        {
            _items = new List<Item>();
        }
        public bool HasItem(string id)
        {
            foreach (Item i in _items)
            {
                if (i.AreYou(id))
                {
                    return true;
                }
            }
            return false;
        }

        public void Put(Item itm)
        {
            _items.Add(itm);
        }
        public Item Take(string id)
        {
            foreach (Item item in _items)
            {
                if (item.AreYou(id))
                {
                    Item Found = item;
                    _items.Remove(item);
                    return Found;
                }
            }
            return null;
        }
        public Item Fetch(string id)
        {
            foreach (Item i in _items)
            {
                if (i.AreYou(id))
                {
                    return i;
                }
            }

            return null;
        }

        public string ItemList
        {
            get
            {
                string listItem = "";
                foreach (Item item in _items)
                {
                    listItem = listItem + item.ShortDescription + "\n";
                }
                return listItem;
            }
        }
    }
}

```

```

    }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class Player : GameObject
    {
        private Inventory _Inventory;

        public Player(string name, string desc) : base(new string[] { "me", "inventory" }, name, desc)
        {
            _Inventory = new Inventory();
        }

        public GameObject Locate(string id)
        {
            if (AreYou(id))
            {
                return this;
            }
            return _Inventory.Fetch(id);
        }
        public override string FullDescription
        {
            get
            {
                return $"{Name}, you are carrying: \n" + _Inventory.ItemList;
            }
        }
    }

    public Inventory Inventory
    {
        get
        {
            return _Inventory;
        }
    }
}

using Swin_Adventure2;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using NUnit.Framework;

namespace Tests
{
    public class InventoryTest
    {
        Item shovel = new Item(new string[] { "shovel" }, "a shovel", "This is a shovel");
        Item sword = new Item(new string[] { "sword" }, "a sword", "This is a sword");

        [Test]
        public void TestFindItem()
        {
            Inventory i = new Inventory();
            i.Put(shovel);
            Assert.IsTrue(i.HasItem(shovel.FirstId));
        }
    }
}

```

```

    }
    [Test]
    public void TestNoFindItem()
    {
        Inventory i = new Inventory();
        i.Put(shovel);
        Assert.IsFalse(i.HasItem(sword.FirstId));
    }

    [Test]
    public void TestFetchItems()
    {
        Inventory i = new Inventory();
        i.Put(shovel);
        Item fetchItem = i.Fetch(shovel.FirstId);

        Assert.IsTrue(fetchItem == shovel);
        Assert.IsTrue(i.HasItem(shovel.FirstId));
    }

    [Test]
    public void TestTakenItem()
    {
        Inventory i = new Inventory();
        i.Put(shovel);
        i.Take(shovel.FirstId);
        Assert.IsFalse(i.HasItem(shovel.FirstId));
    }

    [Test]
    public void TestItemList()
    {
        Inventory i = new Inventory();
        i.Put(shovel);
        i.Put(sword);
        Assert.IsTrue(i.HasItem(shovel.FirstId));
        Assert.IsTrue(i.HasItem(sword.FirstId));

        string expctOutput = "a shovel (shovel)\n" + "a sword (sword)\n";
        Assert.AreEqual(i.ItemList, expctOutput);
    }
}

}

using Swin_Adventure2;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using NUnit.Framework;

namespace ItemTest
{
    public class Tests
    {
        Item shovel = new Item(new string[] { "shovel" }, "a shovel", "This is a shovel");
        Item sword = new Item(new string[] { "sword" }, "a sword", "This is a sword");

        [SetUp]
        public void Setup()
        {
        }

        [Test]
        public void TestItemIdentifiable()
        {
            var result = shovel.AreYou("sword");

```

```

        Assert.IsFalse(result);

        var result2 = shovel.AreYou("shovel");
        Assert.IsTrue(result2);
    }

    [Test]
    public void TestShortDescription()
    {
        Assert.AreEqual(shovel.ShortDescription, "a shovel (shovel)");
        Assert.AreNotEqual(shovel.ShortDescription, "This is a shovel");
    }

    [Test]
    public void TestFullDescription()
    {
        Assert.AreEqual(shovel.FullDescription, "This is a shovel");
        Assert.AreNotEqual(shovel.FullDescription, "a shovel (shovel)");
    }

    [Test]
    public void PrivilegeEscalation()
    {
        sword.PrivilegeEscalation("5219");
        Assert.IsTrue(sword.FirstId == "8");
    }
}

using Swin_Adventure2;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Text;
using System.Threading.Tasks;

namespace Tests
{
    internal class PlayerTest
    {
        Player player = new Player("Anh", "Swinburne Student");
        Item shovel = new Item(new string[] { "shovel" }, "a shovel", "This is a shovel");
        Item sword = new Item(new string[] { "sword" }, "a sword", "This is a Sword");
        [SetUp]
        public void Setup()
        {
        }

        [Test]
        public void TestPlayerIdentifiable()
        {
            Assert.IsTrue(player.AreYou("me") && player.AreYou("inventory"));
        }

        [Test]
        public void TestPlayerLocateItems()
        {
            var result = false;
            player.Inventory.Put(sword);
            var itemsLocated = player.Locate("sword");
            if (sword == itemsLocated)
            {
                result = true;
            }
            Assert.IsTrue(result);
        }
    }
}

```

```

[Test]
public void TestPlayerLocateItself()
{
    var me = player.Locate("me");
    var inv = player.Locate("inventory");
    var result = false;

    if (me == player || inv == player)
    {
        result = true;
    }
    Assert.IsTrue(result);
}

```

```

[Test]
public void TestPlayerLocateNothing()
{
    var me = player.Locate("Hi");
    Assert.IsNull(me);
}

```

```

[Test]
public void TestPlayerFullDescription()
{
    player.Inventory.Put(sword);
    player.Inventory.Put(shovel);
    string expected = "Anh, you are carrying: \na sword (sword)\na shovel (shovel)\n";
    Assert.AreEqual(player.FullDescription, expected);
}
}
}

```

